```
!pip install transformers
```

```
Requirement already satisfied: transformers in /usr/local/lib/pythor
Requirement already satisfied: filelock in /usr/local/lib/python3.10
Requirement already satisfied: huggingface-hub<1.0,>=0.23.0 in /usr/
Requirement already satisfied: numpy>=1.17 in /usr/local/lib/python3
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/pyt
Requirement already satisfied: pyyaml>=5.1 in /usr/local/lib/python3
Requirement already satisfied: regex!=2019.12.17 in /usr/local/lib/p
Requirement already satisfied: requests in /usr/local/lib/python3.10
Requirement already satisfied: tokenizers<0.20,>=0.19 in /usr/local/
Requirement already satisfied: safetensors>=0.4.1 in /usr/local/lib/
Requirement already satisfied: tqdm>=4.27 in /usr/local/lib/python3.
Requirement already satisfied: fsspec>=2023.5.0 in /usr/local/lib/py
Requirement already satisfied: typing-extensions>=3.7.4.3 in /usr/lo
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/loca
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/pythor
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/
```

```
from transformers import VisionEncoderDecoderModel, ViTFeatureExtra
import torch
from PIL import Image


model = VisionEncoderDecoderModel.from_pretrained("nlpconnect/vit-g
feature_extractor = ViTFeatureExtractor.from_pretrained("nlpconnect
tokenizer = AutoTokenizer.from_pretrained("nlpconnect/vit-gpt2-imag

device = torch.device("cuda" if torch.cuda.is_available() else "cpu
model.to(device)



max_length = 16
num_beams = 4
gen_kwargs = {"max_length": max_length, "num_beams": num_beams}
def predict_step(image_paths):
  images = []
  for image_path in image_paths:
    i_image = Image.open(image_path)
    if i_image.mode != "RGB":
      i_image = i_image.convert(mode="RGB")

    images.append(i_image)

  pixel_values = feature_extractor(images=images, return_tensors="p
  pixel_values = pixel_values.to(device)

  output_ids = model.generate(pixel_values, **gen_kwargs)

  preds = tokenizer.batch_decode(output_ids, skip_special_tokens=Tr
  preds = [pred.strip() for pred in preds]
  return preds
  predict_step(['sample2.jpg']) # ['a woman in a hospital bed with
```

```
from transformers import VisionEncoderDecoderModel, ViTFeatureExtra
import torch
from PIL import Image
```

```
model = VisionEncoderDecoderModel.from_pretrained("nlpconnect/vit-g
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
  warnings.warn(
config.json: 100% 4.61k/4.61k [00:00<00:00, 280kB/s]
pytorch_model.bin: 100% 982M/982M [00:11<00:00, 121MB/s]
```

```
feature_extractor = ViTFeatureExtractor.from_pretrained("nlpconnect
tokenizer = AutoTokenizer.from_pretrained("nlpconnect/vit-gpt2-imag
```

```
preprocessor_config.json: 100% 228/228 [00:00<00:00, 4.42kB/s]
/usr/local/lib/python3.10/dist-packages/transformers/models/vit/feat
  warnings.warn(
tokenizer_config.json: 100% 241/241 [00:00<00:00, 2.54kB/s]
vocab.json: 100% 798k/798k [00:00<00:00, 2.94MB/s]
merges.txt: 100% 456k/456k [00:00<00:00, 3.95MB/s]
tokenizer.json: 100% 1.36M/1.36M [00:00<00:00, 6.25MB/s]
special_tokens_map.json: 100% 120/120 [00:00<00:00, 2.55kB/s]
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu
model.to(device)
```

```
bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
            (output): ViTSelfOutput(
              (dense): Linear(in_features=768, out_features=768,
bias=True)
              (dropout): Dropout(p=0.0, inplace=False)
            )
          )
          (intermediate): ViTIntermediate(
            (dense): Linear(in_features=768, out_features=3072,
bias=True)
            (intermediate_act_fn): GELUActivation()
          )
          (output): ViTOutput(
```

```
            (dense): Linear(in_features=3072, out_features=768,
bias=True)
            (dropout): Dropout(p=0.0, inplace=False)
          )
          (layernorm_before): LayerNorm((768,), eps=1e-12,
elementwise_affine=True)
          (layernorm_after): LayerNorm((768,), eps=1e-12,
elementwise_affine=True)
        )
      )
    )
    (layernorm): LayerNorm((768,), eps=1e-12,
elementwise_affine=True)
    (pooler): ViTPooler(
      (dense): Linear(in_features=768, out_features=768,
bias=True)
      (activation): Tanh()
    )
  )
  (decoder): GPT2LMHeadModel(
    (transformer): GPT2Model(
      (wte): Embedding(50257, 768)
      (wpe): Embedding(1024, 768)
      (drop): Dropout(p=0.1, inplace=False)
      (h): ModuleList(
        (0-11): 12 x GPT2Block(
          (ln_1): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
          (attn): GPT2Attention(
            (c_attn): Conv1D()
            (c_proj): Conv1D()
            (attn_dropout): Dropout(p=0.1, inplace=False)
            (resid_dropout): Dropout(p=0.1, inplace=False)
          )
          (ln_2): LayerNorm((768,), eps=1e-05,
elementwise_affine=True)
          (crossattention): GPT2Attention(
            (c_attn): Conv1D()
            (q_attn): Conv1D()
            (c_proj): Conv1D()
            (attn_dropout): Dropout(p=0.1, inplace=False)
            (resid_dropout): Dropout(p=0.1, inplace=False)
          )
          (ln_cross_attn): LayerNorm((768,), eps=1e-05,
```

```
max_length = 16
num_beams = 4
gen_kwargs = {"max_length": max_length, "num_beams": num_beams}
```

```
def predict_step(image_paths):
  images = []
  for image_path in image_paths:
    i_image = Image.open(image_path)
    if i_image.mode != "RGB":
      i_image = i_image.convert(mode="RGB")

    images.append(i_image)

  pixel_values = feature_extractor(images=images, return_tensors="pt
  pixel_values = pixel_values.to(device)

  output_ids = model.generate(pixel_values, **gen_kwargs)

  preds = tokenizer.batch_decode(output_ids, skip_special_tokens=Tru
  preds = [pred.strip() for pred in preds]
  return preds
```

```
predict_step(['environment.jpg'])
```

```
['three children are posing for a picture together']
```